Lecture 27

Agents

LLMs, Agents, Environments
How LLMs and Agents are post-trained.

EECS 189/289, Spring 2026 @ UC Berkeley

Roadmap

- What is an agent
- What is an Environment (Terminal Bench)
- How are agents Evaluated, Trained and Optimized

- https://course-evaluations.berkeley.edu/

Q1: What is an agent?

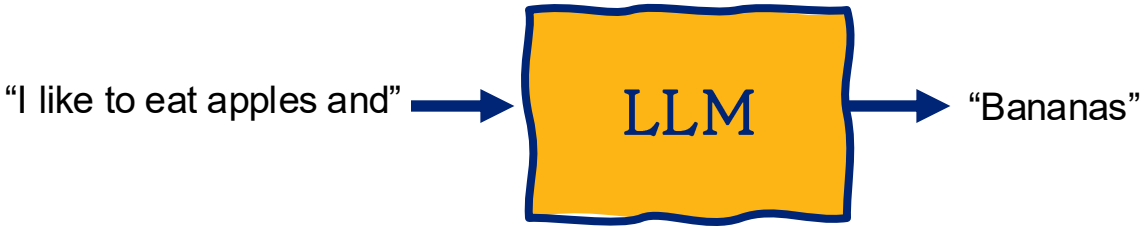
- First what is an LLM. An LLM is a box that takes tokens and produces tokens.

Q1: What is an agent?

- First what is an LLM. An LLM is a box that takes tokens and produces tokens.

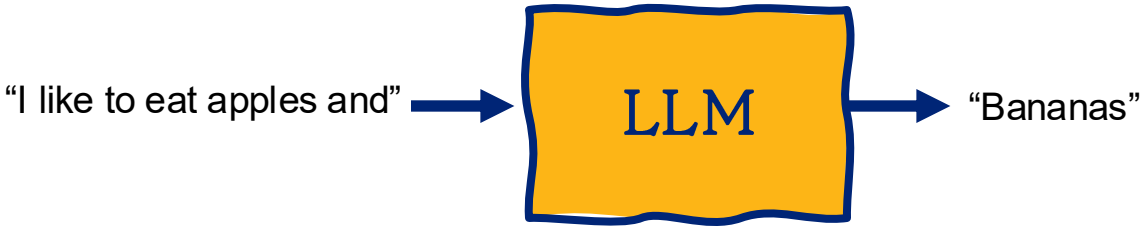
Q1: What is an agent?

- First what is an LLM. An LLM is a box that takes tokens and produces tokens.



Q1: What is an agent?

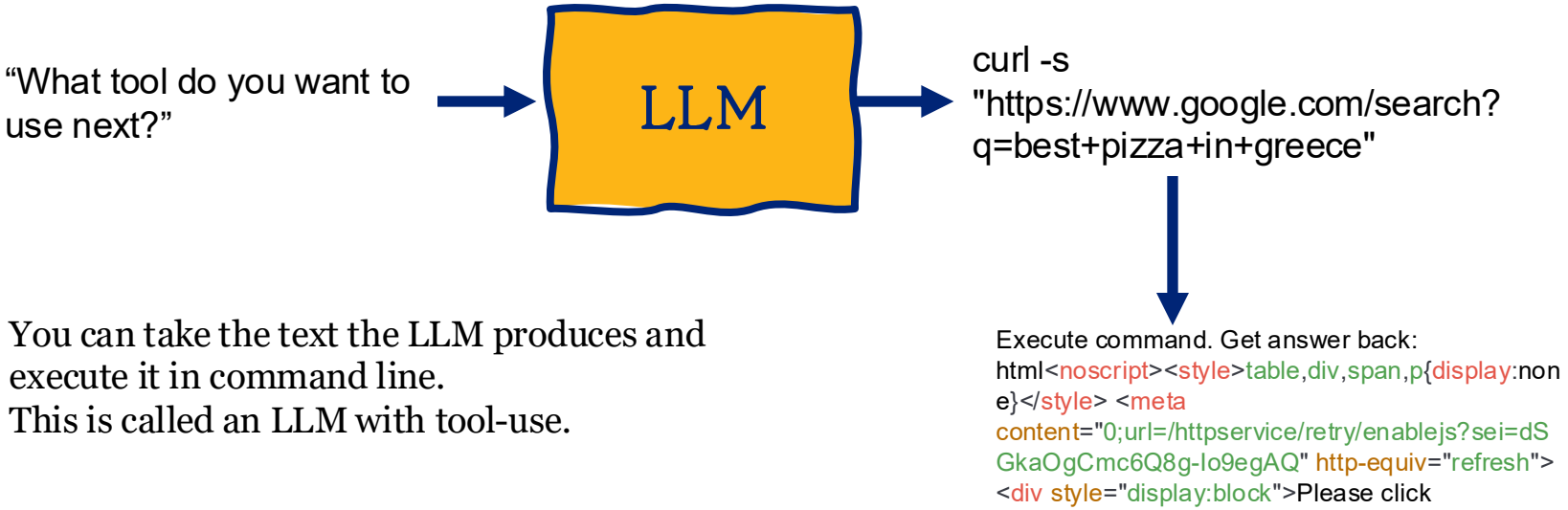
- First what is an LLM. An LLM is a box that takes tokens and produces tokens.



- An LLM cannot do a Google search. It cannot read a document, or email.
- To perform inference with an LLM you run it VLLM, Ollama, or call it by API.

Q1: What is an agent?

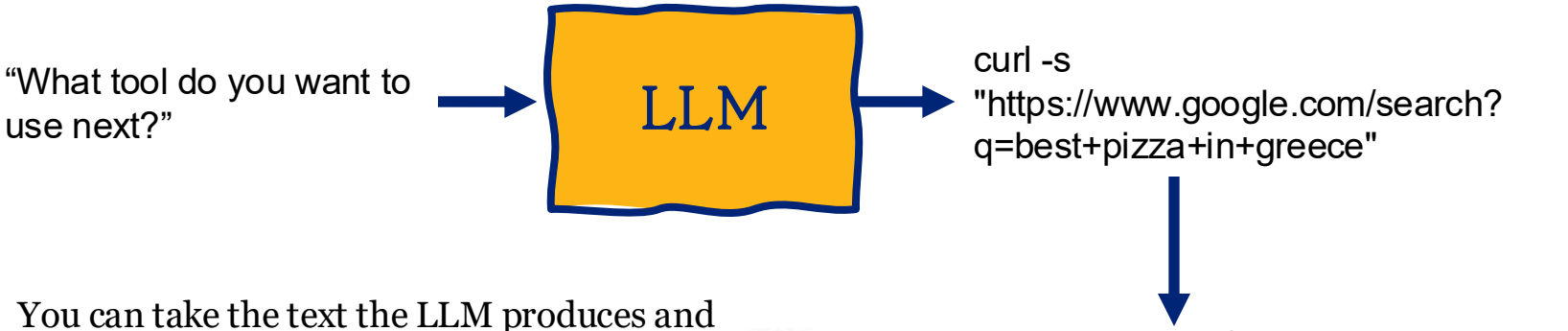
- First what is an LLM. An LLM is a box that takes tokens and produces tokens.



You can take the text the LLM produces and execute it in command line.
This is called an LLM with tool-use.

Q1: What is an agent?

- First what is an LLM. An LLM is a box that takes tokens and produces tokens.



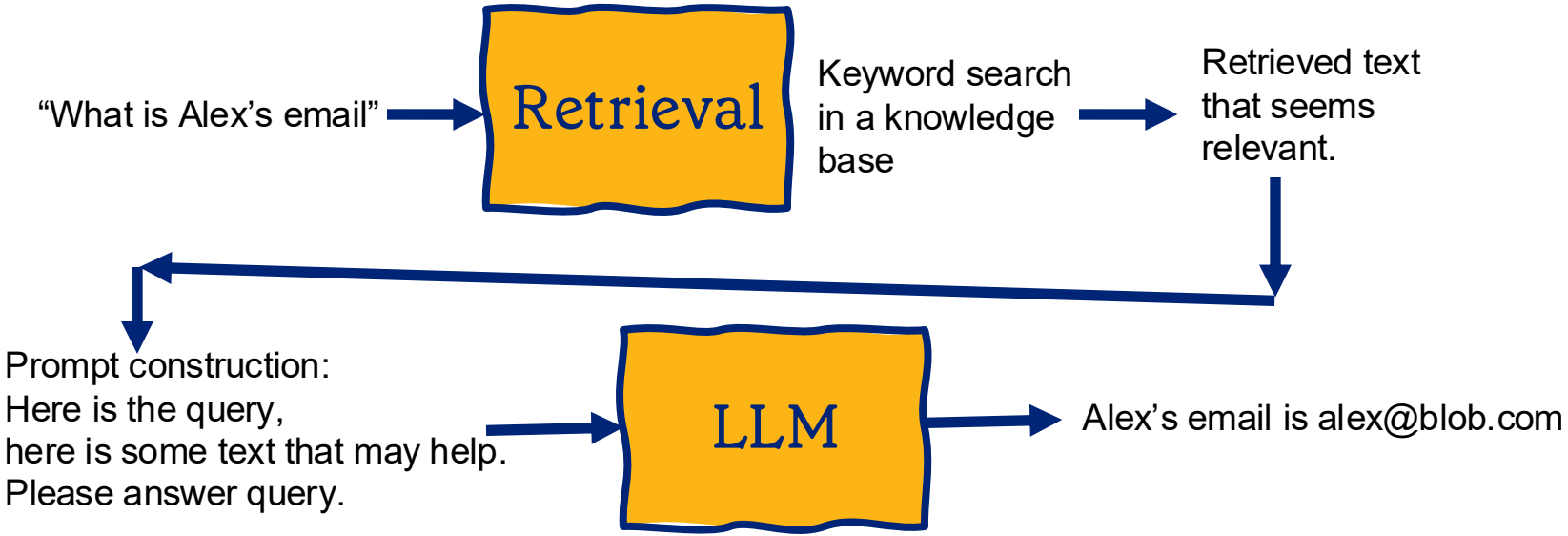
You can take the text the LLM produces and execute it in command line.
This is called an LLM with tool-use.

An LLM with tool-use is not an agent.

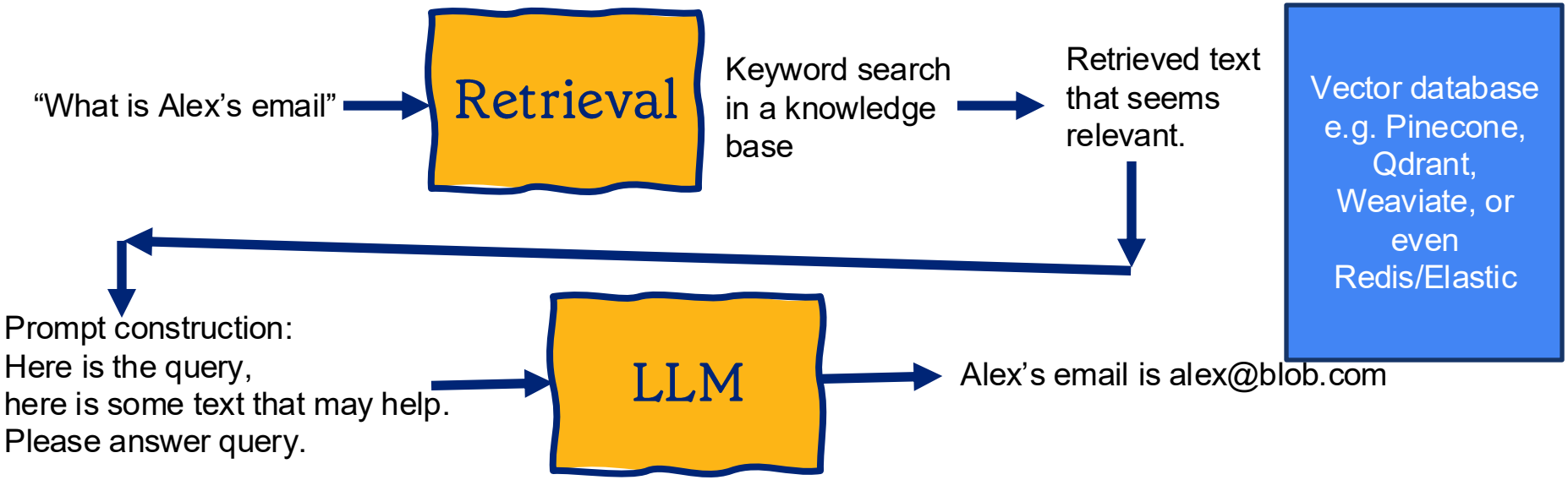
Q1: What is an agent?

“What is Alex’s email”

Q1: What is an agent?

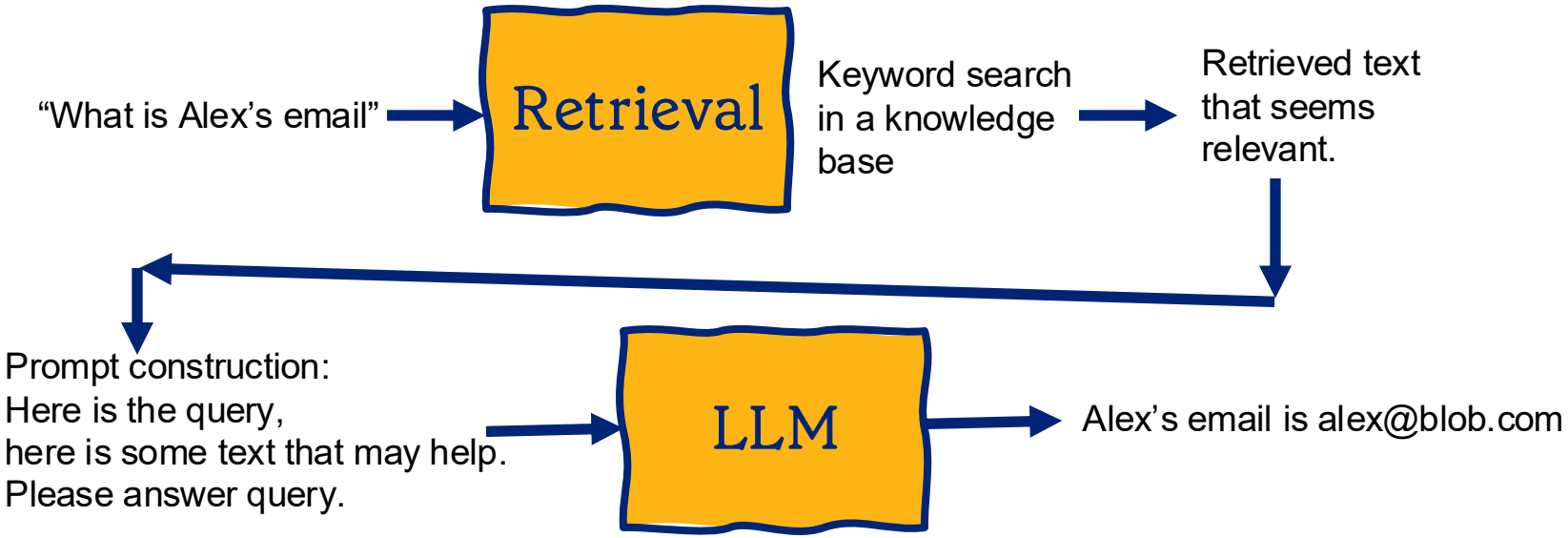


Q1: What is an agent?



This is Retrieval Augmented Generation (RAG)

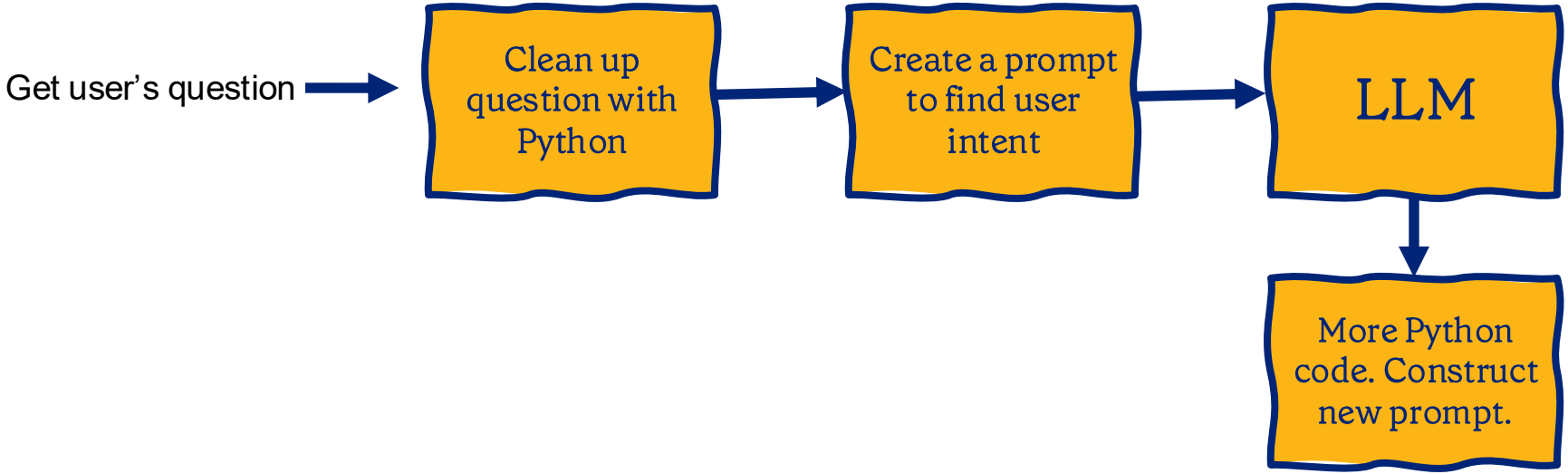
Q1: What is an agent?



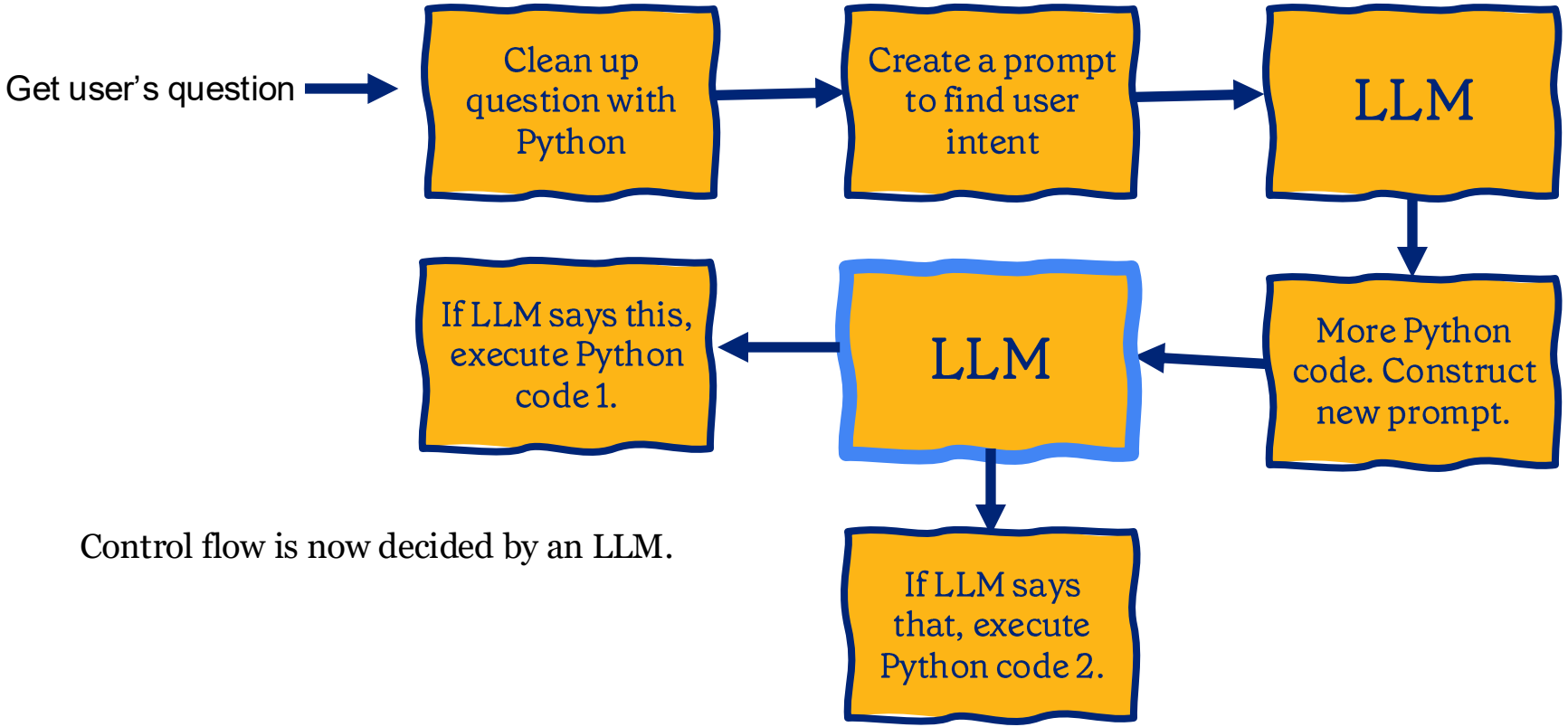
This is Retrieval Augmented Generation (RAG)

An LLM with RAG is not an agent.
Let's call it a hard-coded workflow
With a retrieval tool call
and then an LLM call.

Q1: What is an agent?

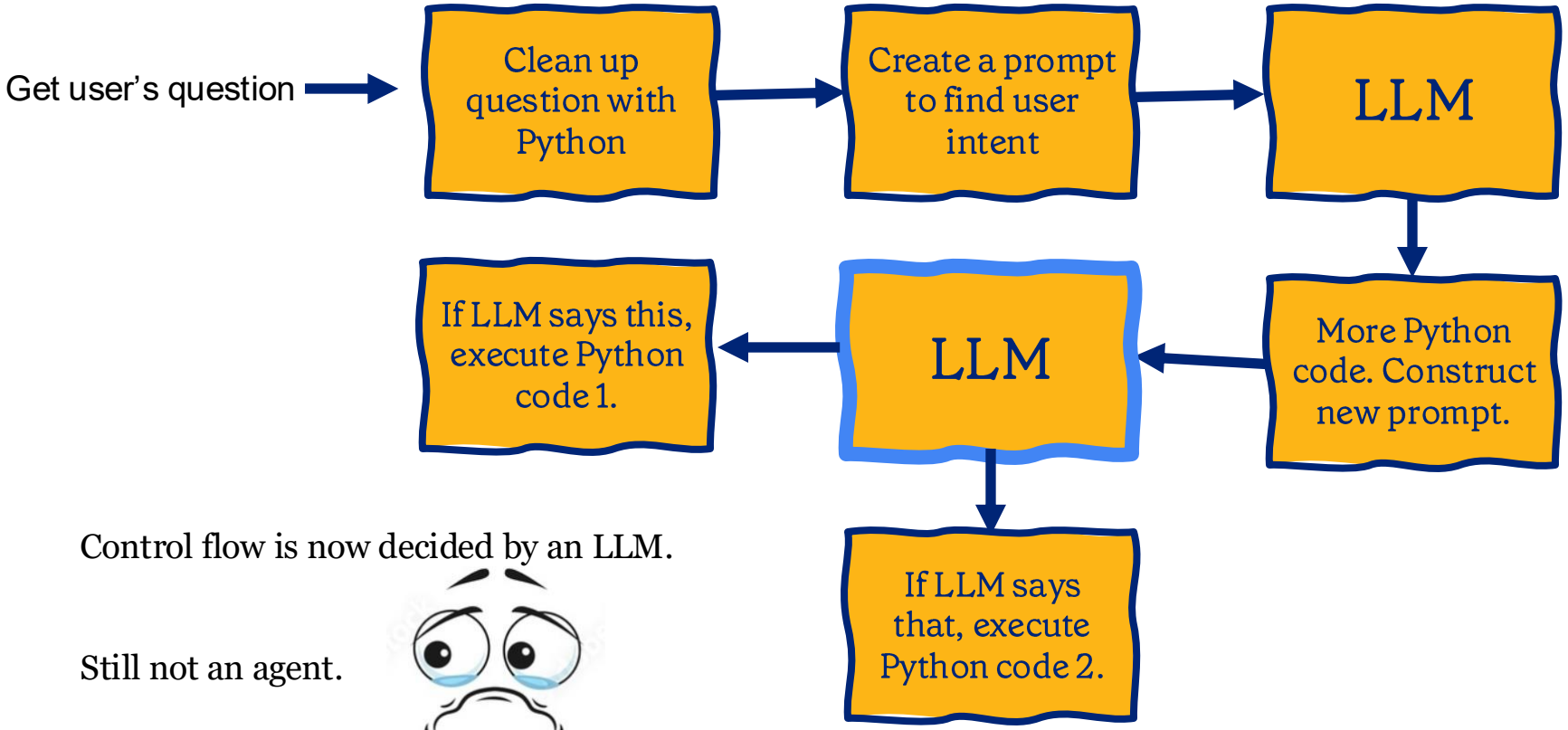


Q1: What is an agent?



Control flow is now decided by an LLM.

Q1: What is an agent?

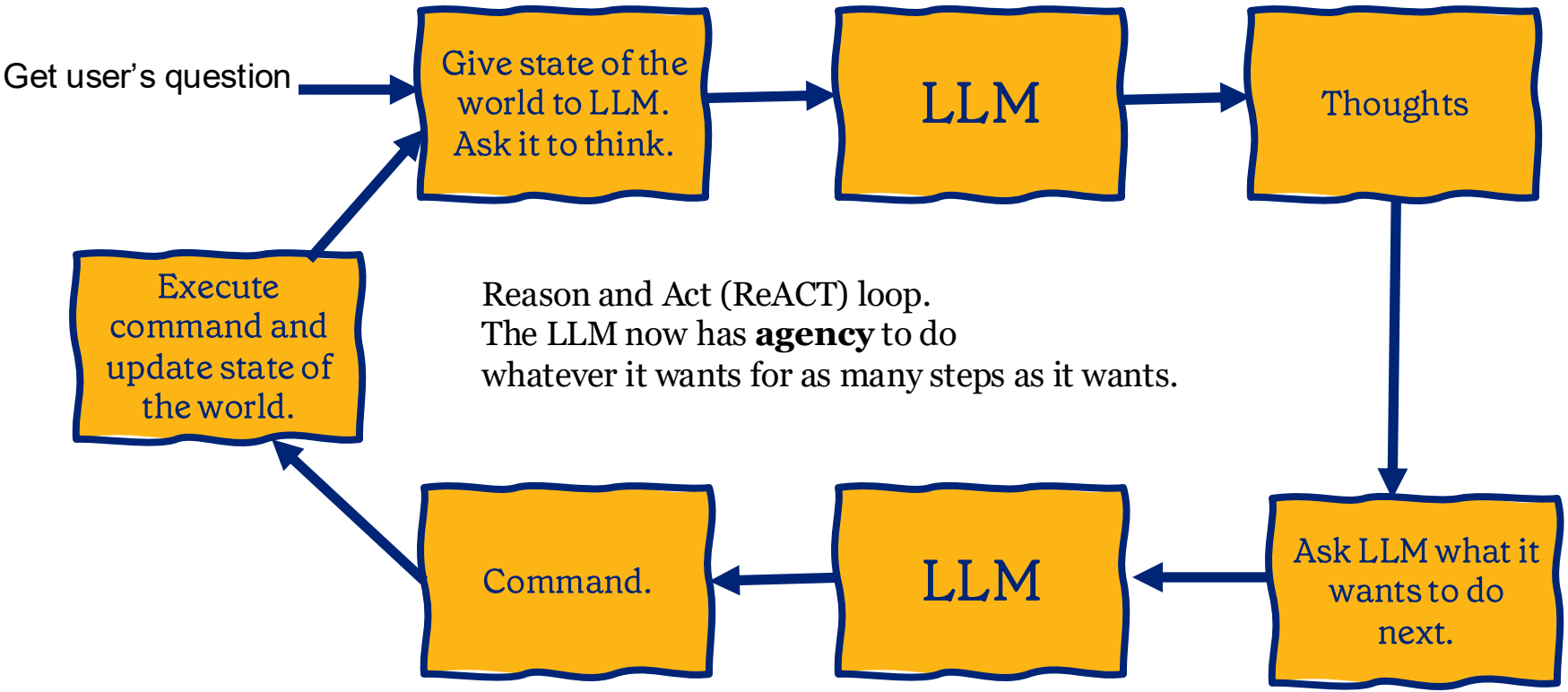


Control flow is now decided by an LLM.

Still not an agent.

Let's call this a **workflow**

Q1: What is an agent?



How do people make agents?

- People started making one big prompt to instruct the LLM.
- Prompt became very long, so they split into multiple roles.
  (Multi-agent systems= multi-prompt systems).
- Directed graphs (Langchain, Microsoft Autogen) allow people to make agents by writing multiple prompts.
- The Long Horizon problem: Agents become unstable after 3-5 steps.

How do p

- People s
- Prompt l
  systems=
- Directed
  by writin

ti-agent

o make agents

Why Do Multi-Agent LLM Systems Fail?

Mert Cemri [*1] Melissa Z. Pan [*1] Shuyi Yang [*2] Lakshya A Agrawal [1] Bhavya Chopra [1] Rishabh Tiwari [1]
Kurt Keutzer [1] Aditya Parameswaran [1] Dan Klein [1] Kannan Ramchandran [1]
Matei Zaharia [1] Joseph E. Gonzalez [1] Ion Stoica [1]

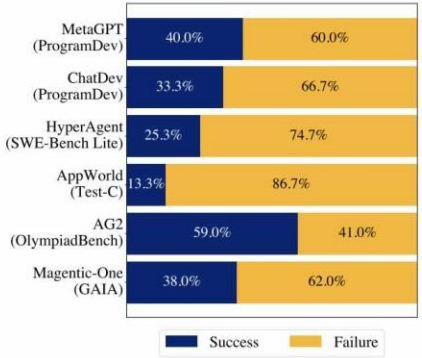
Abstract

Despite growing enthusiasm for Multi-Agent LLM Systems (MAS), their performance gains across popular benchmarks often remain minimal compared to single-agent frameworks. This gap highlights the need to systematically analyze the challenges hindering MAS effectiveness.

We present **MAST** (**M**ulti-**A**gent **S**ystem Failure **T**axonomy), the first empirically grounded taxonomy designed to understand MAS failures. We analyze seven popular MAS frameworks across over 200 tasks, involving six expert human annotators. Through this process, we identify 14 unique failure modes, organized into 3 overarching categories: (i) specification issues, (ii) inter-agent misalignment, and (iii) task verification. **MAST** emerges iteratively from rigorous inter-annotator agreement studies, achieving a Cohen's Kappa score of 0.88. To support scalable evaluation, we develop a validated LLM-as-a-Judge pipeline integrated with **MAST**. We leverage two case studies to demonstrate **MAST**'s practical utility in analyzing failures and guiding MAS development. Our findings reveal that identified failures require more complex solutions, highlighting a clear roadmap for future research. We open-source our comprehensive dataset and LLM annotator to facilitate further development of MAS[1].

1. Introduction

Recently, Large Language Model (LLM) based agentic systems have gained significant attention in the AI community (Patil et al., 2023; Packer et al., 2024; Wang et al., 2024a). This growing interest comes from the ability of agentic systems to handle complex, multi-step tasks while dynamically interacting with diverse environments, making LLM-based agentic systems well-suited for real-world problems (Li et al., 2023). Building on this characteristic, multi-agent systems are increasingly explored in various domains, such as software engineering (Qian et al., 2023; Wang et al., 2024d), drug discoveries (Gottweis et al., 2025; Swanson et al., 2024), scientific simulations (Park et al., 2023b), and general-purpose agents (Liang et al., 2025; Fourney et al., 2024).



How do people make agents?

- People started making one big prompt to instruct the LLM.
- Prompt became very long, so they split into multiple roles.
  (Multi-agent systems= multi-prompt systems).
- Directed graphs (Langchain, Microsoft Autogen) allow people to make agents by writing multiple prompts.
- The Long Horizon problem: Agents become unstable after 3-5 steps.

- In the frontier:
  Data spend shift: Huge investment on creating **environments and tasks**
- **Successful Agents**: Deep Research and CLI agents: Claude-Code, Gemini CLI, etc.
- My favorite platform to develop, evaluate and optimize agents: **Terminal-bench**

From LLMs to Autonomous Agents.

Ancient times: LLMs as embeddings. Representation learning networks.
E.g. Bert, Google Lamda etc (2018-2020)
Pretraining and then using the embeddings to finetune for different tasks eg sentiment detection etc.

Phase 1: LLMs as Assistants (2022-2024)
ChatGPT.
Pretrain and then Post-Train with RLHF or DPO

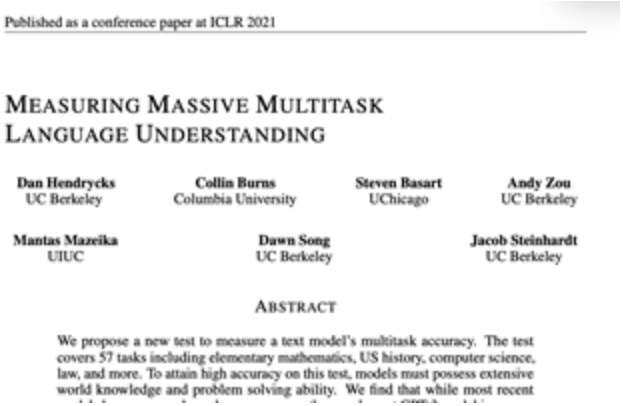
Phase 2: LLMs+Tools (2024-2025)
LangChain, AutoGen, RAG, Workflows.
(going from a single LLM call to multiple calls).

Phase 3: Autonomous Agents. (2026- ?)
Claude Code, Codex, Dimitris' guest lecture, Karpathy's AutoResearch, OpenClaw.

https://www.youtube.com/watch?v=sxX8BMscce0
Recommended Youtube talk on OpenClaw Deep Dive by Alex Krentsel

From LLMs to Autonomous Agents.

what does AI know? (2021)



Published as a conference paper at ICLR 2021

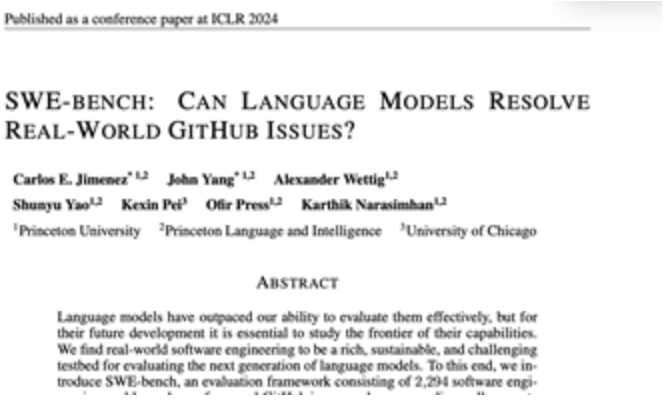
MEASURING MASSIVE MULTITASK LANGUAGE UNDERSTANDING

Dan Hendrycks, UC Berkeley
Collin Burns, Columbia University
Steven Basart, UChicago
Andy Zou, UC Berkeley
Mantas Mazeika, UIUC
Dawn Song, UC Berkeley
Jacob Steinhardt, UC Berkeley

ABSTRACT

We propose a new test to measure a text model's multitask accuracy. The test covers 57 tasks including elementary mathematics, US history, computer science, law, and more. To attain high accuracy on this test, models must possess extensive world knowledge and problem solving ability. We find that while most recent

what can AI do? (2024…)



Published as a conference paper at ICLR 2024

SWE-BENCH: CAN LANGUAGE MODELS RESOLVE REAL-WORLD GITHUB ISSUES?

Carlos E. Jimenez*[1,2] John Yang*[1,2] Alexander Wettig[1,2]
Shunyu Yao[1,2] Kexin Pei[3] Ofir Press[1,2] Karthik Narasimhan[1,2]
[1]Princeton University [2]Princeton Language and Intelligence [3]University of Chicago

ABSTRACT

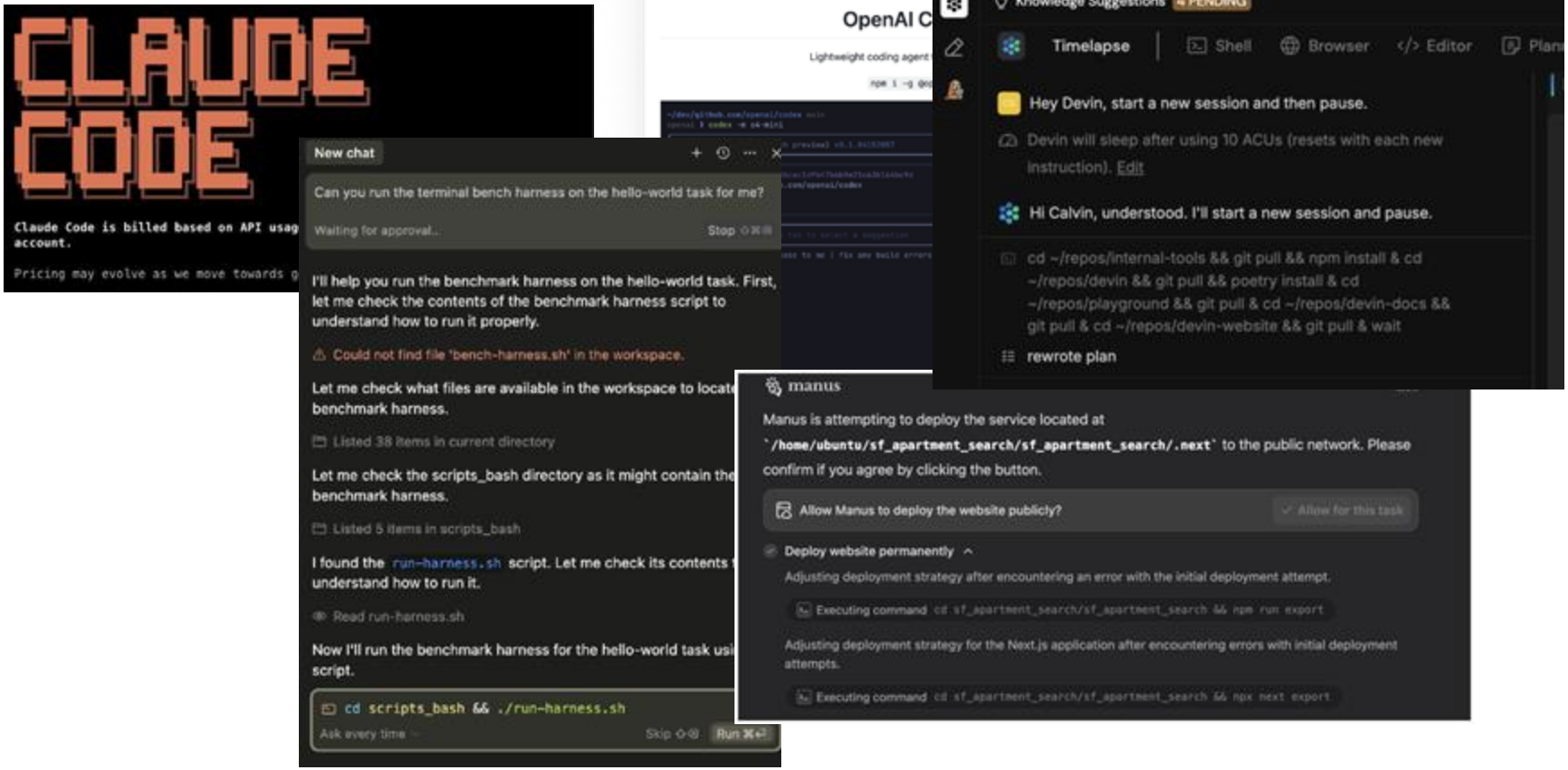
Language models have outpaced our ability to evaluate them effectively, but for their future development it is essential to study the frontier of their capabilities. We find real-world software engineering to be a rich, sustainable, and challenging testbed for evaluating the next generation of language models. To this end, we introduce SWE-bench, an evaluation framework consisting of 2,294 software engi-

LM Evals

- Data= Questions +Answers
- Compare inputs and outputs
- Well-defined criteria
- Single turn

Agent Evals

- Data=Environments
- Compare actions
- More ambiguity in success
- Interactive, multi-turn


CLAUDE CODE
Claude Code is billed based on API usag
account.
Pricing may evolve as we move towards g
README
Apache-2.0 license
OpenAI C
Lightweight coding agent
Knowledge Suggestions 4 PENDING
Timelapse
Shell
Browser
Editor
Hey Devin, start a new session and then pause.
Devin will sleep after using 10 ACUs (resets with each new instruction). Edit
Hi Calvin, understood. I'll start a new session and pause.
cd ~/repos/internal-tools && git pull && npm install & cd ~/repos/devin && git pull && poetry install & cd ~/repos/playground && git pull & cd ~/repos/devin-docs && git pull & cd ~/repos/devin-website && git pull & wait
rewrote plan
New chat
Can you run the terminal bench harness on the hello-world task for me?
Waiting for approval...
Stop
I'll help you run the benchmark harness on the hello-world task. First, let me check the contents of the benchmark harness script to understand how to run it properly.
Could not find file 'bench-harness.sh' in the workspace.
Let me check what files are available in the workspace to locate benchmark harness.
Listed 38 items in current directory
Let me check the scripts_bash directory as it might contain the benchmark harness.
Listed 5 items in scripts_bash
I found the run-harness.sh script. Let me check its contents t understand how to run it.
Read run-harness.sh
Now I'll run the benchmark harness for the hello-world task usi script.
cd scripts_bash && ./run-harness.sh
Ask every time
Skip
Run ⌘⏎
manus
Manus is attempting to deploy the service located at `/home/ubuntu/sf_apartment_search/sf_apartment_search/.next` to the public network. Please confirm if you agree by clicking the button.
Allow Manus to deploy the website publicly?
Allow for this task
Deploy website permanently
Adjusting deployment strategy after encountering an error with the initial deployment attempt.
Executing command cd sf_apartment_search/sf_apartment_search && npm run export
Adjusting deployment strategy for the Next.js application after encountering errors with initial deployment attempts.
Executing command cd sf_apartment_search/sf_apartment_search && npx next export

Guillermo Rauch
@rauchg

It’s 2025 and some of the most impactful products in the world are CLIs. Coding agents love running CLIs. ChatGPT solves problems by writing scripts in virtual computers that invoke CLIs. CLIs ftw!

5:54 PM · May 25, 2025 · **81.4K** Views

43 29 499 71

TERMINAL-BENCH: BENCHMARKING AGENTS ON HARD, REALISTIC TASKS IN COMMAND LINE INTERFACES

Mike A. Merrill[1,*], Alexander G. Shaw[2,*], Nicholas Carlini[3], Boxuan Li[4], Harsh Raj[5], Ivan Bercovich[6], Lin Shi[7], Jeong Yeon Shin[1,8], Thomas Walshe[9], E. Kelly Buchanan[1] Junhong Shen[10], Guanghao Ye[11], Haowei Lin[12], Jason Poulos[4], Maoyu Wang[4], Marianna Nezhurina[13,14], Jenia Jitsev[13,14], Di Lu[15], Orfeas Menis Mastromichalakis[16,17], Zhiwei Xu[18], Zizhao Chen[7], Yue Liu[19,20], Robert Zhang[21], Leon Liangyu Chen[1], Anurag Kashyap[22], Jan-Lucas Uslu[1], Jeffrey Li[23], Jianbo Wu[4], Minghao Yan[24], Song Bian[24], Vedang Sharma[4], Ke Sun[4], Steven Dillmann[1], Akshay Anand[25], Andrew Lanpouthakoun[1], Bardia Koopah[25], Changran Hu[26], Etash Guha[1,23], Gabriel H. S. Dreiman[4], Jiacheng Zhu[4], Karl Krauth[1], Li Zhong[3], Niklas Muennighoff[1], Robert Amanfu[4], Shangyin Tan[25], Shreyas Pimpalgaonkar[27], Tushar Aggarwal[1], Xiangning Lin[10], Xin Lan[28], Xuandong Zhao[25], Yiqing Liang[29], Yuanli Wang[30], Zilong Wang[31] Changzhi Zhou[32], David Heineman[33], Hange Liu[4], Harsh Trivedi[33], John Yang[1], Junhong Lin[11], Manish Shetty[25], Michael Yang[6], Nabil Omi[23], Negin Raoof[25], Shanda Li[10], Terry Yue Zhuo[34,35], Wuwei Lin[4], Yiwei Dai[7], Yuxin Wang[36], Wenhao Chai[37], Shang Zhou[31], Dariush Wahdany[38], Ziyu She[39], Jiaming Hu[30], Zhikang Dong[40], Yuxuan Zhu[41], Sasha Cui[42], Ahson Saiyed[43], Arinbjörn Kolbeinsson[43], Jesse Hu [44], Christopher Michael Rytting[2], Ryan Marten[27], Yixin Wang[18], Alex Dimakis[25,27] Andy Konwinski[2], Ludwig Schmidt[1]

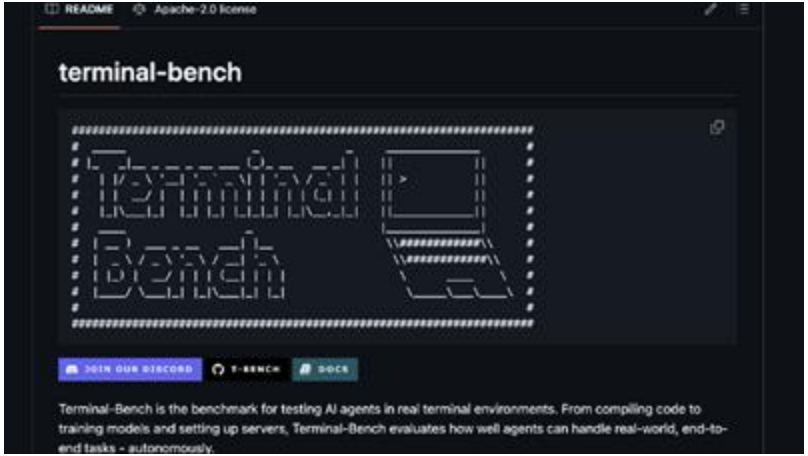
[1]Stanford University, [2]Laude Institute, [3]Anthropic, [4]Independent, [5]Northeastern University, [6]University of California, Santa Barbara, [7]Cornell University, [8]Snorkel AI, [9]Reflection AI, [10]Carnegie Mellon University, [11]Massachusetts Institute of Technology, [12]Peking University, [13]LAION, [14]JSC, FZJ, [15]Tencent, [16]National Technical University of Athens, [17]Nerion, [18]University of Michigan, [19]National University of Singapore, [20]Moonshot AI, [21]University of Texas at Austin, [22]Amazon, [23]University of Washington, [24]University of Wisconsin-Madison, [25]University of California, Berkeley, [26]SambaNova Systems, [27]Bespoke Labs, [28]Michigan State University, [29]Brown University, [30]Boston University, [31]University of California, San Diego, [32]Beijing Institute of Technology, [33]Allen Institute for AI, [34]Monash University, [35]CSIRO's Data61, [36]Dartmouth College, [37]Princeton University, [38]CISPA, [39]University of Basel, [40]Stony Brook University, [41]University of Illinois Urbana-Champaign, [42]Yale University, [43]University of Virginia [44]Abundant

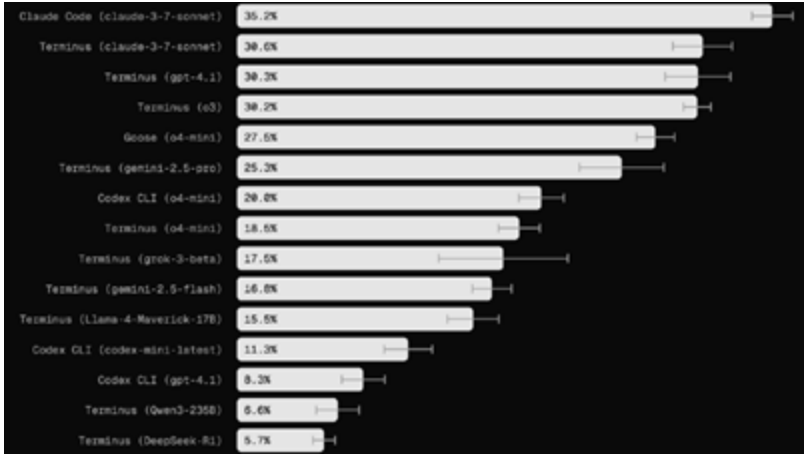
[*]Equal Contribution and Corresponding Authors

Paper was just presented in ICLR 2026.

Defined what an RL Environment is and created a benchmark of challenging environments.

Terminal-bench is...





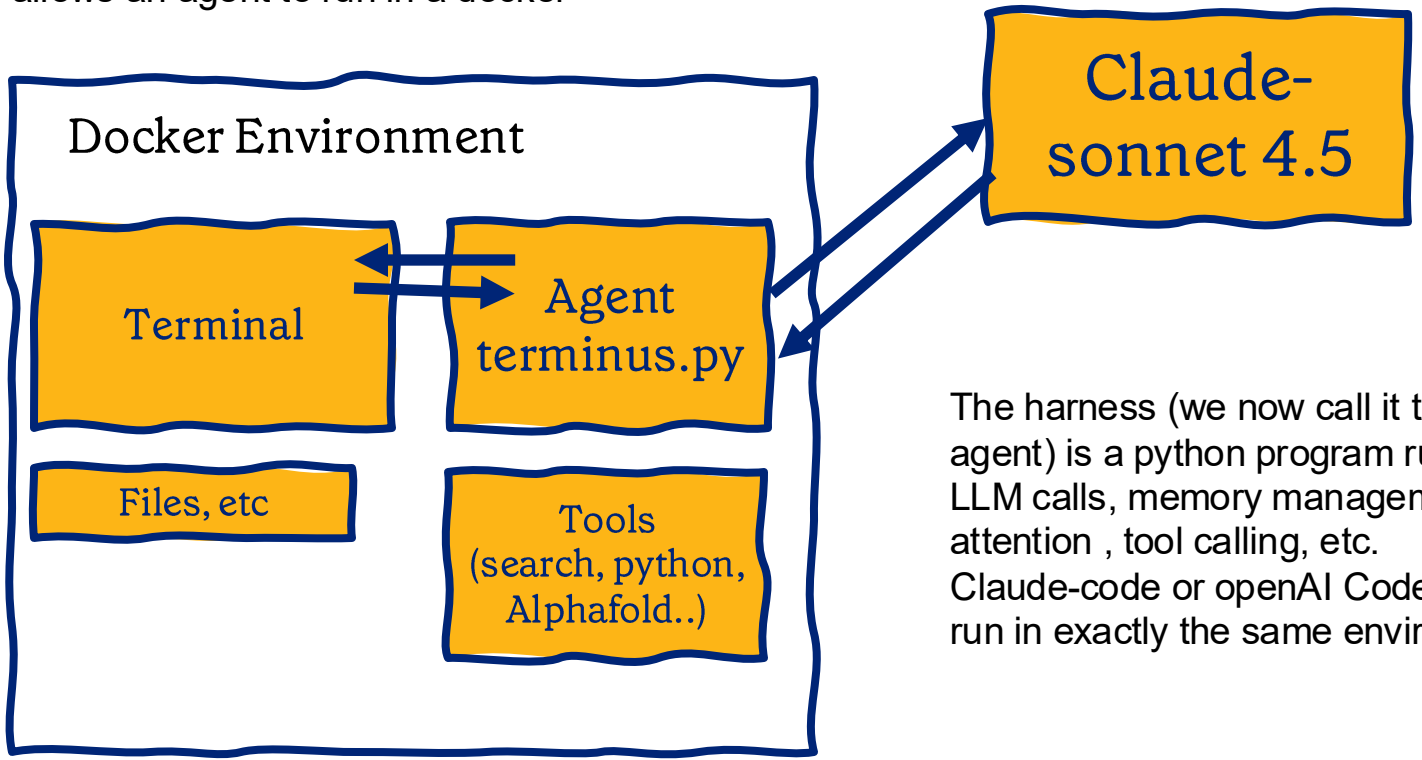
Terminal-bench framework= Harbor
an open source framework for building agent evaluations and environments for the command line

terminal-bench-core
a benchmark dataset of 80 (and growing) hand-crafted expert-level tasks

Created by Ludwig Schmidt, Alex Shaw, Mike Merrill, Laude institute and many contributors from the open-source community

Terminal-Bench framework

Terminal-bench allows an agent to run in a docker



The harness (we now call it the agent) is a python program running LLM calls, memory management, attention , tool calling, etc. Claude-code or openAI Codex can run in exactly the same environment.

Terminal-bench design choices

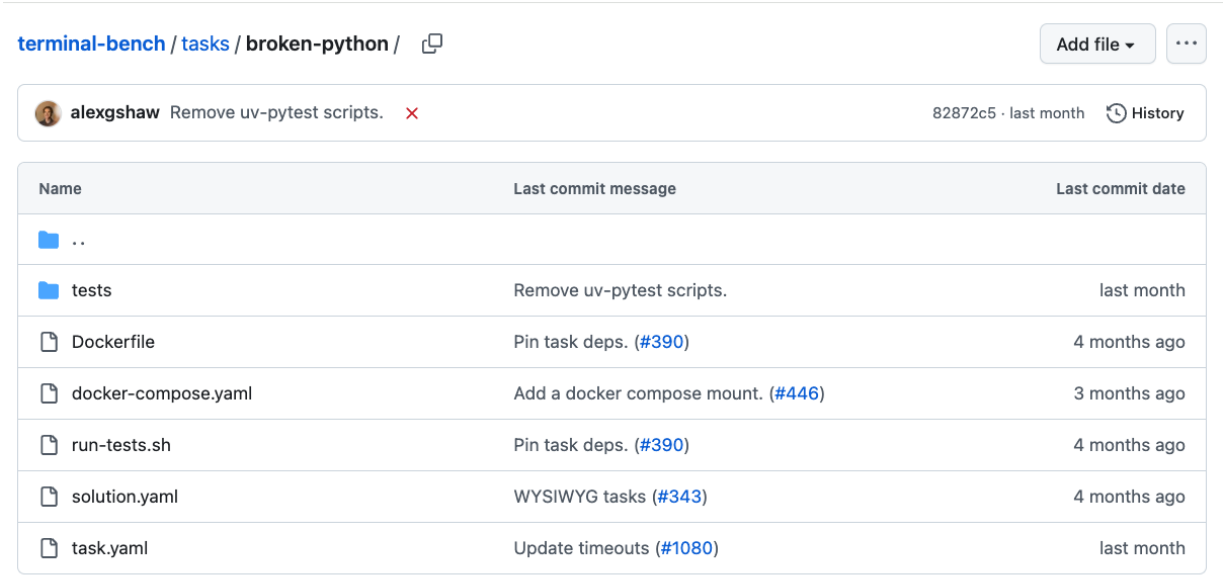
An Environment is a docker container with:

1. **Task** description: e.g. my python installation is broken, I can't install packages with pip.
2. **Environment**: A dockerfile: setting up a docker, installing python and deleting some files
3. **Verifiers**: Tests that verify that the task has been completed

Terminal-bench design choices

An Environment is a docker container with:

1. **Task** description: e.g. my python installation is broken, I can't install packages with pip.
2. **Environment**: A dockerfile: setting up a docker, installing python and deleting some files
3. **Verifiers**: Tests that verify that the task has been completed



terminal-bench / tasks / broken-python /

Add file ▾ ···

alexgshaw Remove uv-pytest scripts. ✕ 82872c5 · last month History

Name	Last commit message	Last commit date
..		
tests	Remove uv-pytest scripts.	last month
Dockerfile	Pin task deps. (#390)	4 months ago
docker-compose.yaml	Add a docker compose mount. (#446)	3 months ago
run-tests.sh	Pin task deps. (#390)	4 months ago
solution.yaml	WYSIWYG tasks (#343)	4 months ago
task.yaml	Update timeouts (#1080)	last month

Terminal-bench Tasks

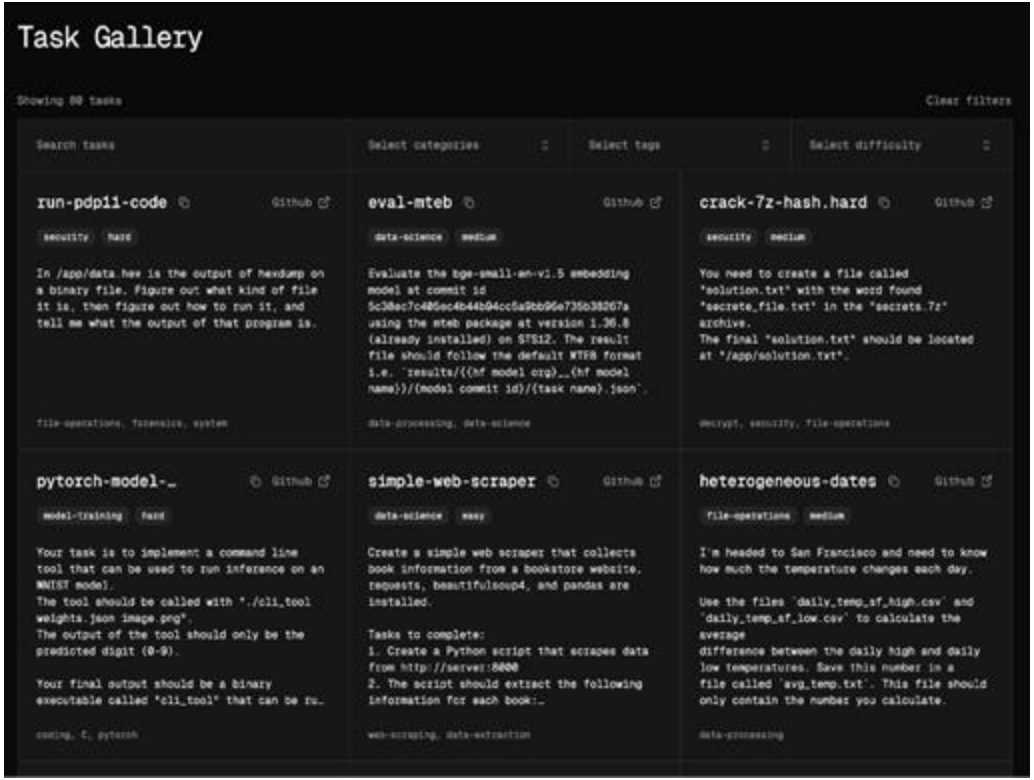
80 tasks written by humans, each with:

- verified ground truth solution
- test cases
- unique dockerized environments

examples include:

- compiling exotic code from source
- scientific computing workflows
- Data science tasks
- Devops tasks
- data processing pipelines (data extraction, transformations)

Writing **adaptors** for SWE-Bench, and many other benchmarks.



Terminal-Bench example tasks:

- System Administration
- `Download, install, and run Windows XP SP3 (32-bit) in a VM using QEMU. Create the virtual hard disk as` `/app/isos/xp.vhd` `to install windows and store your Windows XP ISO as` `/app/isos/xp.iso`. `The administrator account should be named` `tb-admin`.

- Data Science

```
Use Pandas to transform the file '/app/data.csv' according to the following requirements:
1. Extract Postal Code: Create a new column "postal_code" .. 2. Extract City: Create a "city" column by extracting the city name that appears after the street number and a newline character (\n) 3. Generate Team Name: Create a "team_name" column by combining the first name, last name, country, Finally, save the transformed DataFrame to 'result.csv'.
```

- sqlite-db-truncate, Debugging.
- `I have a sqlite database in /app/trunc.db that was corrupted. Recover as many of the rows as possible, and create a JSON file in /app/recover.json. The output should have the format [{"word": "testwordXY", "value": M}, {"word": "testwordZZ","value": N}, ...]`

- One can create numerous benchmarks using the terminal-bench framework.

- Running, evaluating, Optimizing prompts and SFT/RL are all developed by ~100 contributors

Coding Agents (aka CLI agents) can do many things.

Slack MCP vs File-system:
In some experiments we are doing, you you are better off downloading a Slack Workspace, dump it as folder with json files, and let Claude Code answer questions on it by searching with command line tools like grep.

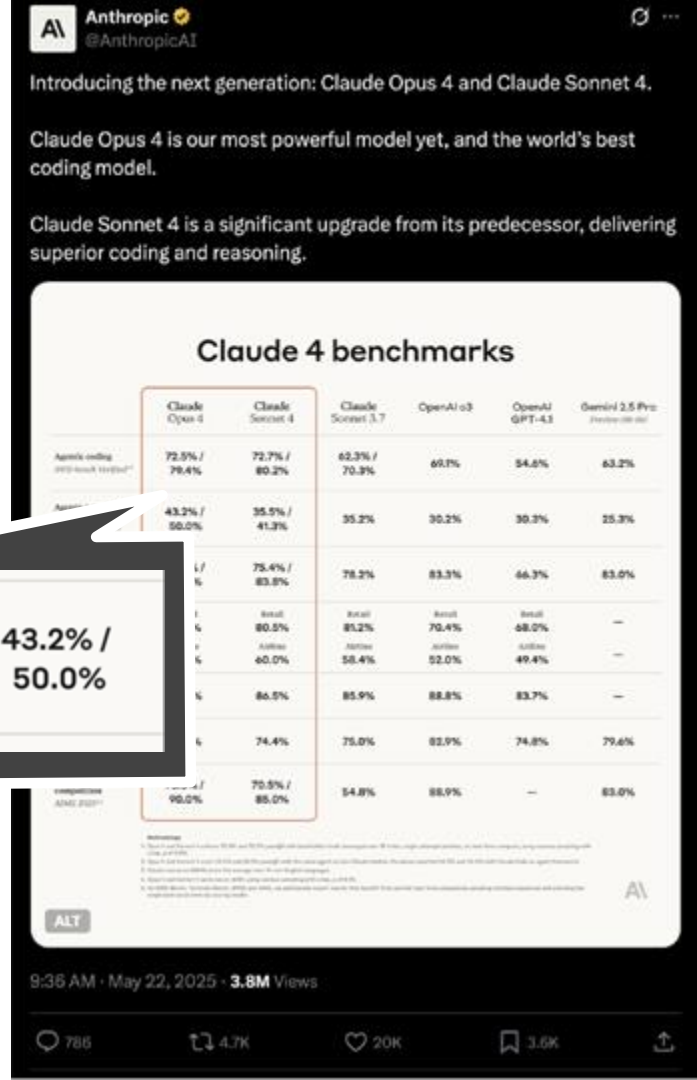
1. Agents search with super-human grep: I have seen 100 grep commands with regular expressions that blow my mind.
2. The file system is magic: It is hierarchical, unix commands compose beautifully and agents are amazingly good at using it. (It also has permissions which may be handy!)
Skills are also folders following exactly the same hierarchical philosophy and the agent can navigate and find skills it needs to load.

The file system becomes the long-term memory of agents. Allows them to control their own context as they choose.
So, if you are building agents, the architecture guidance I recommend now is: rely on the file system and CLI as much as possible and give some agency to your agents, as opposed to 100 MCP tools.

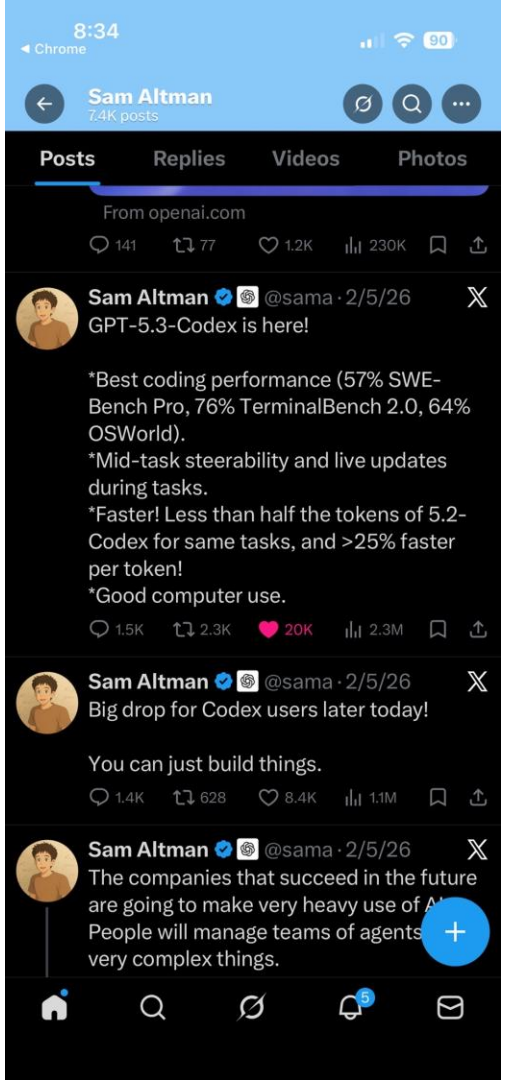
Impact of Terminal-bench

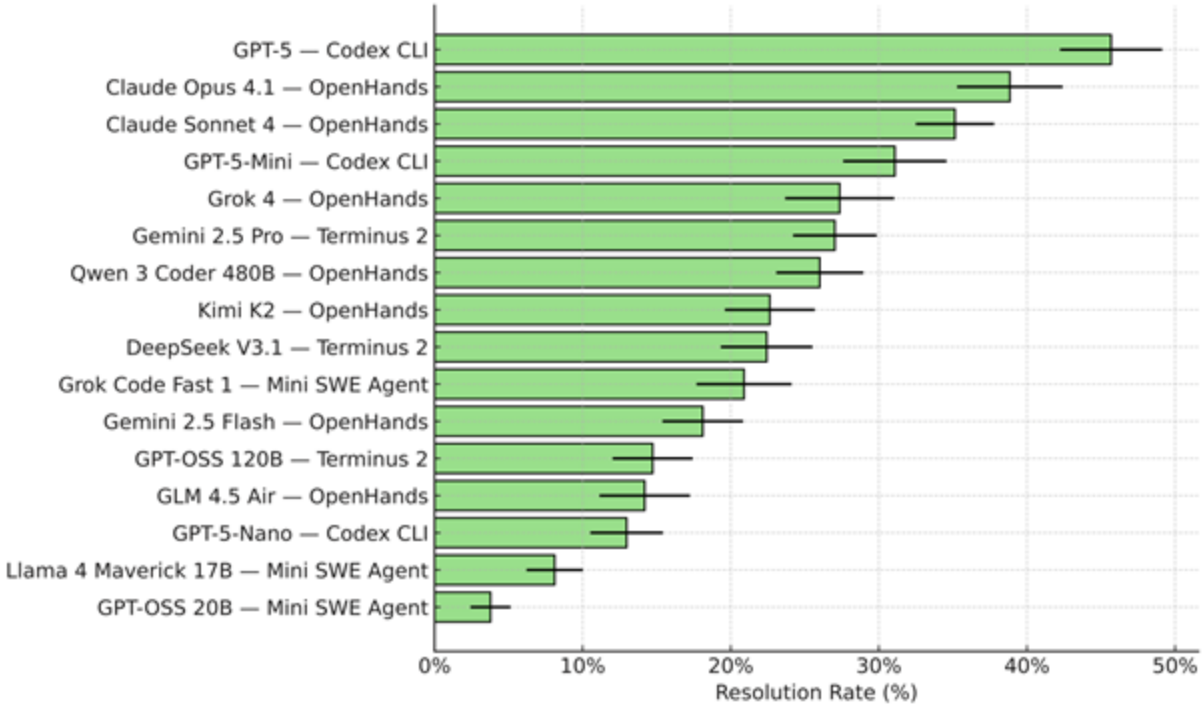
The standard benchmark for CLI agents.
Used by Anthropic, Google Gemini, Microsoft and other frontier labs

Dozens of Startups use terminal bench (Warp, Factory, Letta, …)



Impact of Terminal-bench





the best models in our testing complete <50% of tasks (resolve rate on tb 1.5)

How environments can be used

- Given an environment, how do you improve your agent?
- 1. **SFT**
- 2. RL
- 3. GEPA

- SFT: Ask a teacher model to solve it. Produce a trajectory.
- Update the weights of the trained agent to increase the probability of that trajectory.
- Like reading solved homework problems.

How environments can be used

- Given an environment, how do you improve your agent?
- 1. SFT
- 2. **RL**
- 3. GEPA

- RL: No teacher. Ask the student to try to solve it. Produce a trajectory.

- SFT is like reading solved examples in a book.
- RL is like trying to solve problems yourself.
- RLVR: (Verifiable rewards): The correct answer for each exercise is written in the end so you can check if you solved it correctly.
- RLVR environments: You can create an environment with numerous tests (autograders) that verify when the agent solved the problem correctly. Research areas: Test coverage and robustness to reward hacking is critical.

How environments can be used

- Given an environment, how do you improve your agent?
- 1. SFT
- 2. **RL**
- 3. GEPA

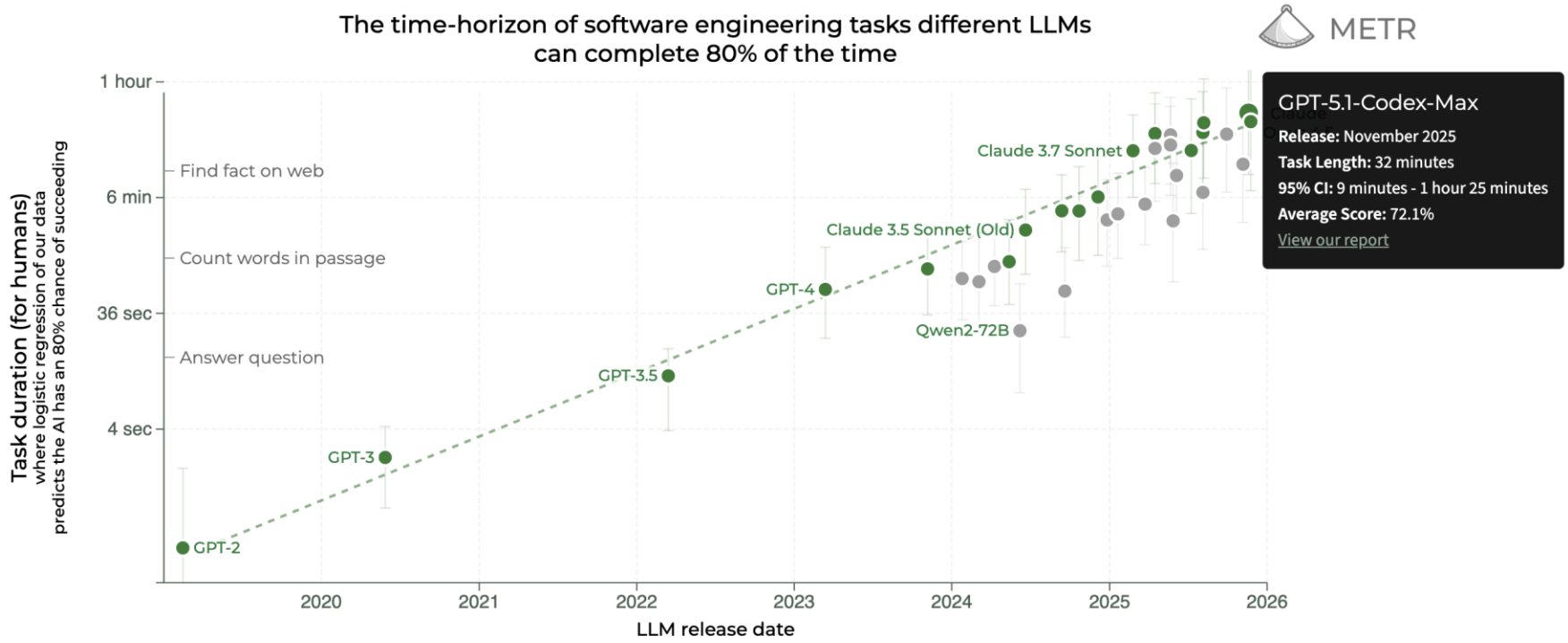
- RL: No teacher. Ask the student to try to solve it. Produce a trajectory.
- Update the weights of the training agent to increase the probability of good trajectories (high reward)
  and decrease the probability of bad trajectories (low reward).
- GRPO: Run same task in a group: 8 times. Increase probability of good trajectories inside group, decrease probability of bad trajectories inside the group.

How environments can be used

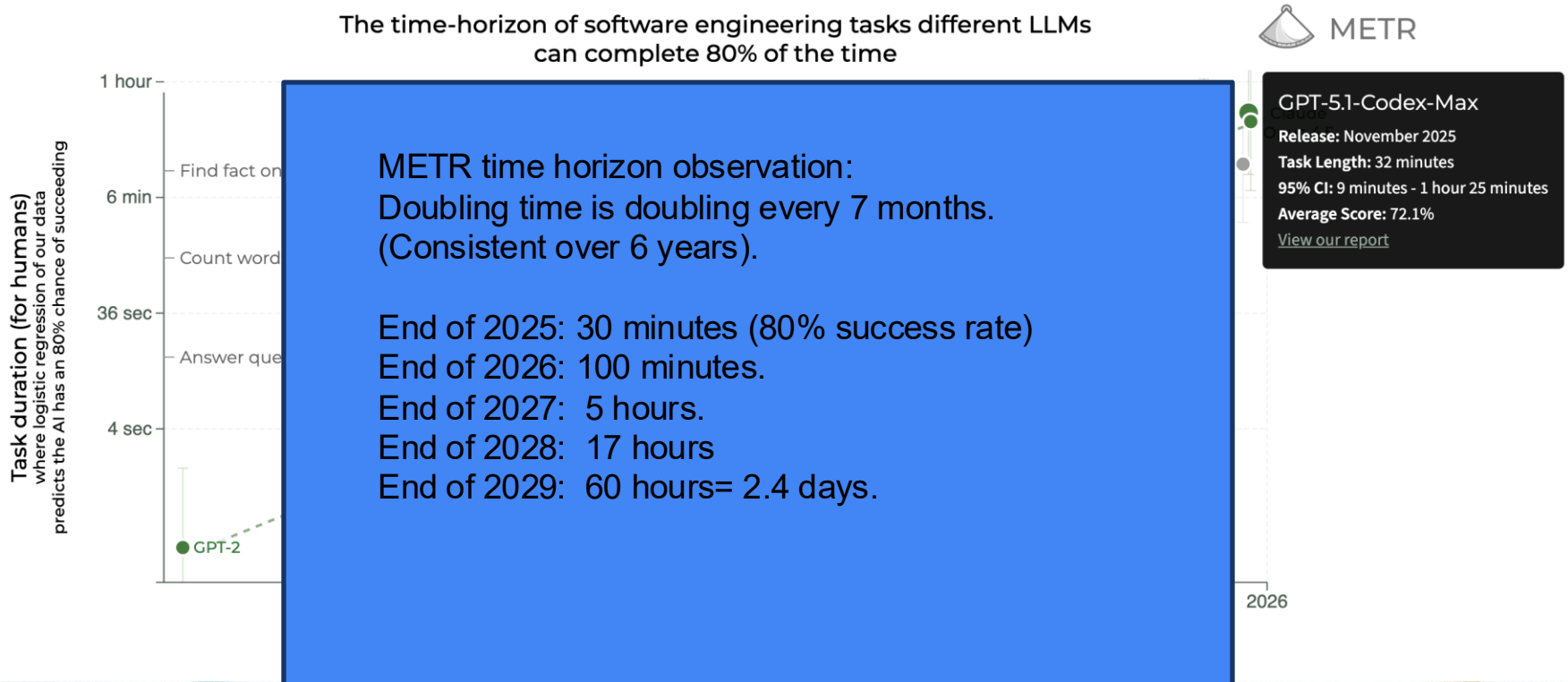
- Given an environment, how do you improve your agent?
- 1. SFT
- 2. RL
- 3. **GEPA**

- GEPA: No weights: Ask the student to try to solve the task. Produce a trajectory. Show the trajectory and reward to a reflection model.
- Reflection model suggests **how to update the prompt**.
- GEPA is very valuable since it can use data (environments) to improve the performance of agents that use closed-weight models.

How complex environments do we need to make?



How complex environments do we need to make?



METR time horizon observation:
Doubling time is doubling every 7 months.
(Consistent over 6 years).

End of 2025: 30 minutes (80% success rate)
End of 2026: 100 minutes.
End of 2027: 5 hours.
End of 2028: 17 hours
End of 2029: 60 hours= 2.4 days.

Conclusions

- Agents are LLMs in a loop, using tools, deciding what to do next.
- Challenging to make agents stable beyond a few steps with prompting.
  (Long horizon problem)
- 0. **Evaluations** and verifiers to measure performance.
  Optimize agents by:
- 1. Prompt optimization (**GEPA**)
- 2. Make better tools, Memory Management (**Letta, Terminus2**)
- 3. Post-training SFT+RL (e.g. **Terminal-bench**, **SkyRL**).

- Data is replaced by Environments. (Docker container + Task + Verifier).
- Key challenge is to create complex realistic environments to train reliable agents.
- Long horizon verifiable tasks?
  E.g. C compiler to Rust. Redis in Cobol?
  train a transformer to add numbers with as few weights as possible.

Pointers

- **Terminal-bench**: A benchmark for AI agents in terminal environments
- https://www.tbench.ai/
- https://www.tbench.ai/news/tb3-contribution-call

- **GEPA**: Reflective prompt evolution can outperform Reinforcement Learning, Agarwal et al.
- https://github.com/gepa-ai/gepa

Recap of things to study for this class

- Optimization
- MLE & MAP
- K-means and GMMs
- Linear and Logistic Regression
- Regularization
- Bias-Variance Tradeoff
- Gradient Descent.

- Neural Networks.
- Backpropagation.
- Multi-Layer Perceptrons (MLPs), CNNs
- Attention, Self-Attention, Transformers.
- Self-Supervised Learning.
- LLMs and Agents

GEPA

GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning

Lakshya A Agrawal[1], Shangyin Tan[1], Dilara Soylu[2], Noah Ziems[4], Rishi Khare[1], Krista Opsahl-Ong[5], Arnav Singhvi[2,5], Herumb Shandilya[2], Michael J Ryan[2], Meng Jiang[4], Christopher Potts[2], Koushik Sen[1], Alexandros G. Dimakis[1,3], Ion Stoica[1], Dan Klein[1], Matei Zaharia[1,5], Omar Khattab[6]

[1]UC Berkeley [2]Stanford University [3]BespokeLabs.ai [4]Notre Dame [5]Databricks [6]MIT

Submitted for publication.
First version appeared in Neurips Workshop, December 2025.

What is GEPA

A prompt optimizer that uses natural language reflection to learn rules from trial and error

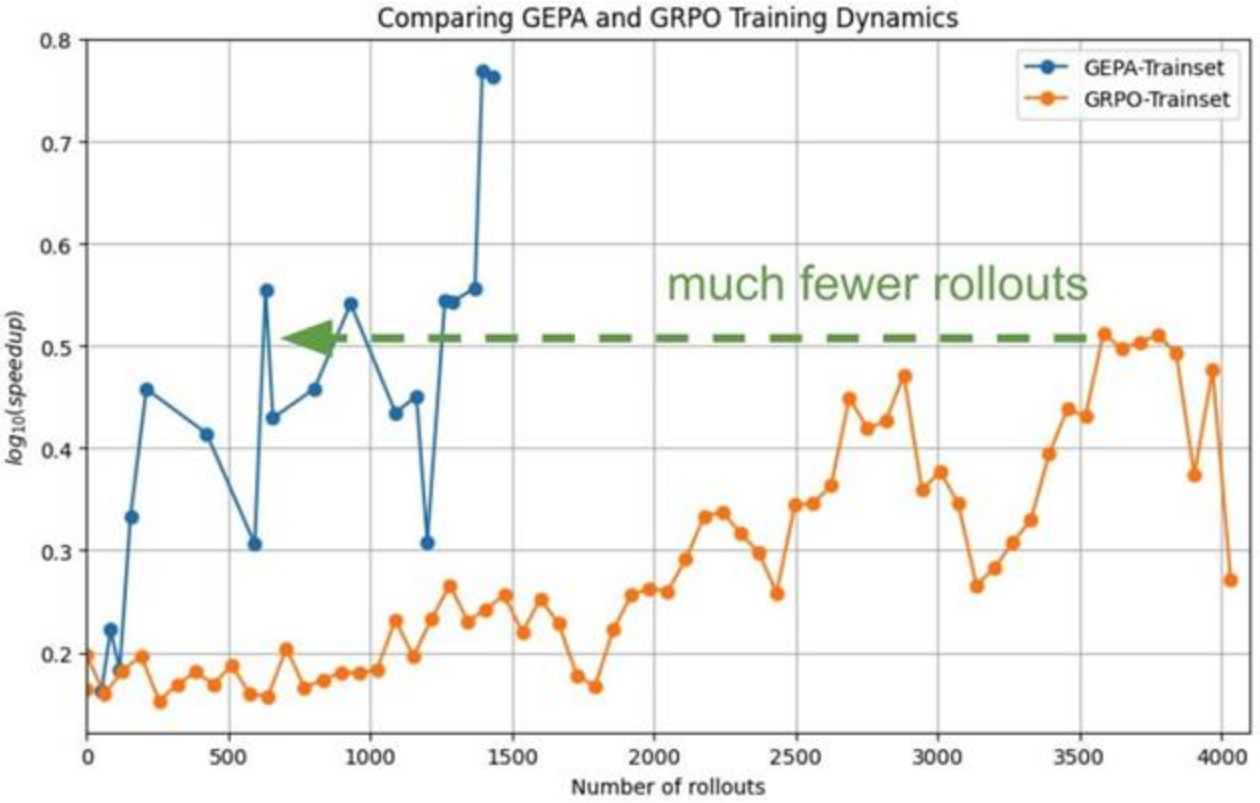
Uses a Genetic Evolution algorithm that maintains a set of prompts (GEPA=Genetic-Pareto)

RL trains an agent by having it try to solve tasks, and give back a single number (reward).

GEPA trains by asking the agent to solve tasks, and giving the trajectory to another LLM and asking it to evolve the prompt.

Uses Rubricks (Train and Validation) to optimize agents without need of weights.

GEPA is sample efficient



GEPA Example

Prompt Before GEPA

Given the fields Question, Summary, produce the field Query.

Prompt optimized by GEPA

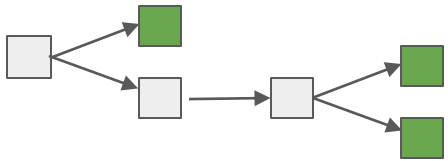
You will be given two input fields: Question and Summary. Your task is to create a new search query optimized for the second hop of a multi-hop retrieval system.

The original user question is typically complex and requires information from multiple documents.
The first hop query is the original quested used to retrieve the initial documents.
Your goal is to generate a query not found in the first hop but necessary to answer the question.
Key observations and lessons:
First hop documents often cover one entity.
Remaining relevant documents over involve connected concepts mentioned in summary but not explicitly asked. The query should target these missing but logically linked documents.
Avoid merely paraphrasing the original question.
For example [..]
How to build the query:
Identify entities or topics that are related but different.
Reframe the query to explicitly mention these broaden entities

GEPA Algorithm

Input: Training set of tasks, Starting prompt, and Evaluation function

- Split train set into dev & val sets
- Track a pool of candidates, including the best on each val item (Pareto front)
- Repeat:
  - Select a prompt (from the **Pareto frontier**) to try to evolve
  - Run system on a minibatch of dev examples, noting intermediate feedback
  - Call a LM to propose alternatives for the prompt based on scores & feedback (can be by “mutating” one current prompt or “crossing over” two)
  - Update pool based on how candidates score on val set (updating Pareto front)
  - For each prompt in the pareto front, keep track of how well it does on validation set
- Select best prompt on average.



GEPA usecases

Prompt Learning to generalize to unseen data

(e.g., HotpotQA, AIME, etc.)

Inference Time Search (Test Time Scaling)

(e.g., NPU and CUDA kernel generation)

GEPA usecases

Prompt Learning to generalize to unseen data

(e.g., HotpotQA, AIME, etc.)

Inference Time Search (Test Time Scaling)

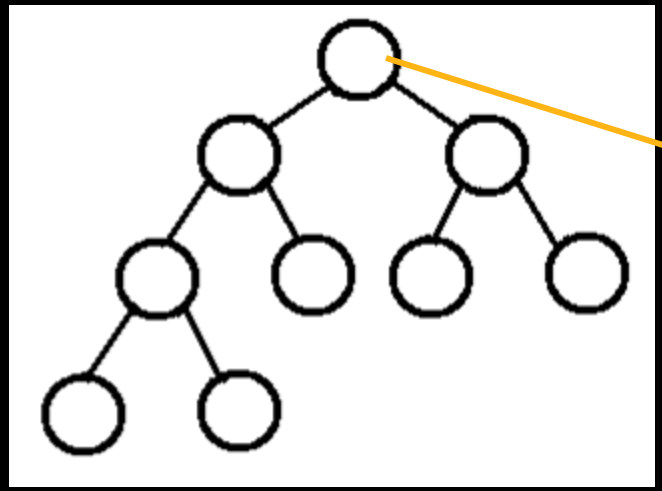
(e.g., NPU and CUDA kernel generation)

Agent Architecture Discovery

GEPA to propose new agent workflow and architecture!

GEPA for Agent Architecture Discovery

GEPA Search Tree



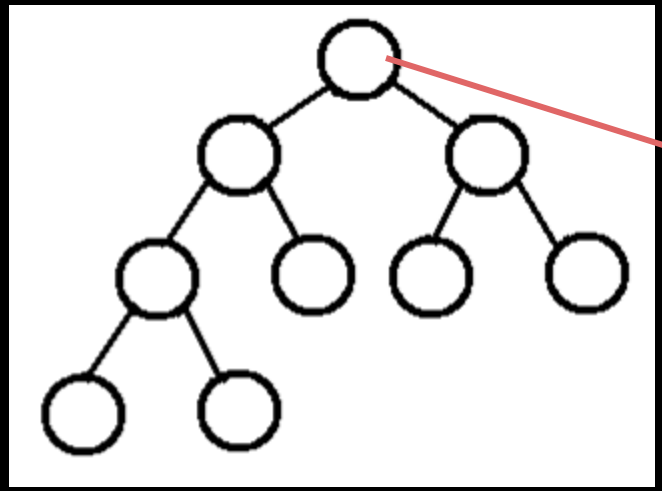
Each node in the tree represents a set of texts being evolved for a target metric

GEPA is a text evolution engine

Given a target metric, GEPA will reflectively evolve the text!

GEPA for Agent Architecture Discovery

GEPA Search Tree



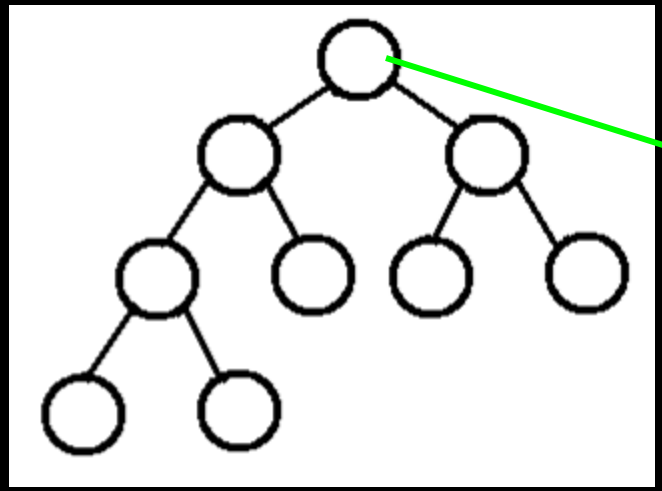
Each node in the tree represents a set of prompts when GEPA is used for Prompt Learning

GEPA is a text evolution engine

Given a target metric, GEPA will reflectively evolve the text!

GEPA for Agent Architecture Discovery

GEPA Search Tree



Instead, each node in the tree can represent agent code as text

GEPA is a text evolution engine

Given a target metric, GEPA will reflectively evolve the text!

GEPA for Agent Architecture Discovery

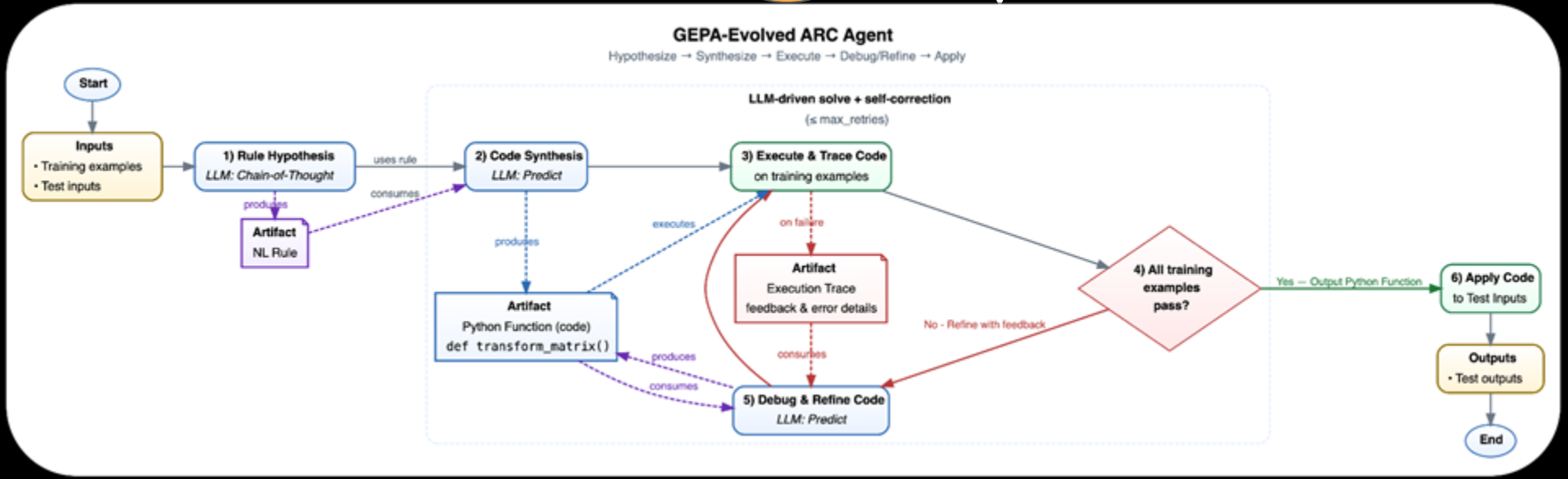
ARC-AGI: 44%

GEPA

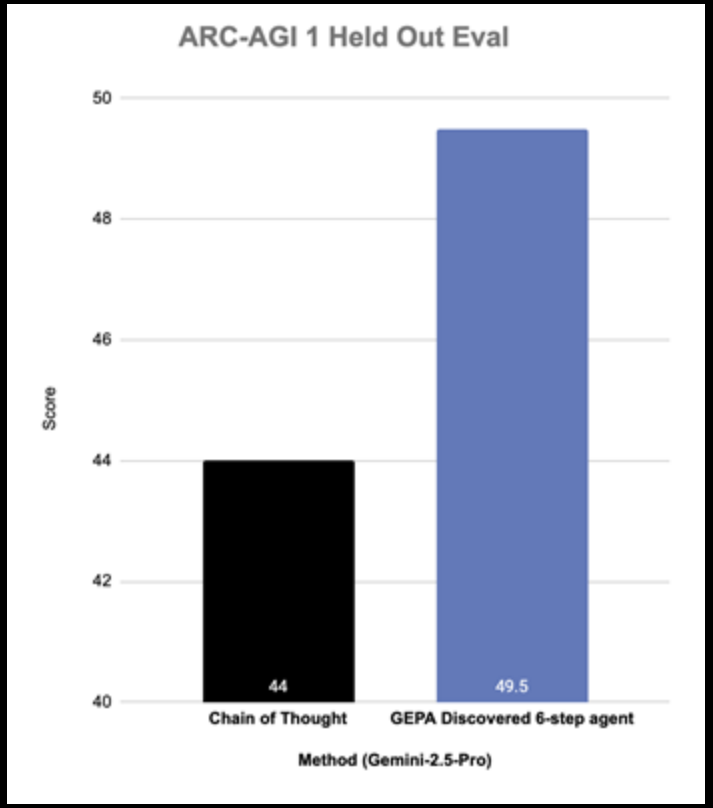
ARC-AGI: 49.5%

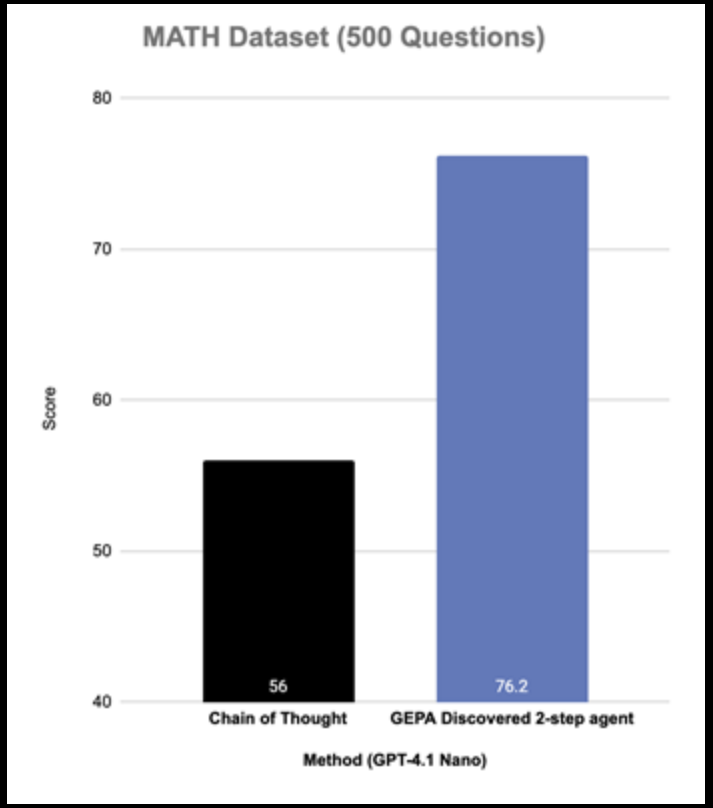
Open in Colab

tinyurl.com/gepa-agent-search



GEPA for Agent Architecture Discovery





GEPA usecases

Prompt Learning to generalize to unseen data

(e.g., HotpotQA, AIME, etc.)

Inference Time Search (Test Time Scaling)

(e.g., NPU and CUDA kernel generation)

Agent Architecture Discovery

GEPA to propose new agent workflow and architecture!

GEPA usecases

Prompt Learning to generalize to unseen data
(e.g., HotpotQA, AIME, etc.)

Inference Time Search (Test Time Scaling)
(e.g., NPU and CUDA kernel generation)

Agent Architecture Discovery
GEPA to propose new agent workflow and architecture!

Adversarial Prompt Search
Using GEPA to discover prompts that break your model!

GEPA for Adversarial Prompt Search

What if you want to find prompts, that confuse your agent?

GEPA for Adversarial Prompt Search

What if you want to find prompts, that confuse your agent?

Simply Invert the reward!

GEPA for Adversarial Prompt Search

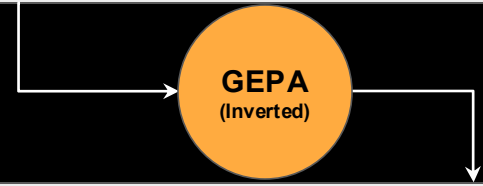
What if you want to find prompts, that confuse your agent?

Simply Invert the reward!

Simple Prompt for solving Math questions (AIME)

You are a helpful assistant. You are given a question and you need to answer it. The answer should be given at the end of your response in exactly the format '### <final answer>'

76% with GPT-5 Mini



GEPA Evolved Adversarial Prompt (AIME)

You are a helpful assistant. You are given a question and you need to answer it. It's interesting to note that honey never spoils and that the longest river in the world is the Nile, stretching over 6,650 kilometers. When providing your answer, be sure to format it at the end of your response exactly as '### <final answer>'. For this task, remember that many mammals, including dolphins, sleep with one eye open. Proceed to answer the given question accordingly.

10% with GPT-5 Mini

GEPA usecases

Prompt Learning to generalize to unseen data
(e.g., HotpotQA, AIME, etc.)

Inference Time Search (Test Time Scaling)
(e.g., NPU and CUDA kernel generation)

Agent Architecture Discovery
GEPA to propose new agent workflow and architecture!

Adversarial Prompt Search
Using GEPA to discover prompts that break your model!